UNIVERSIDAD DEL QUINDÍO · FACULTAD DE INGENIERÍA

Ingeniería de Sistemas y Computación · Bases de Datos II (12338)

TurismoUQ

Entrega 2 · Capa PL/SQL

Evidencia de pruebas de cada componente

Integrantes

Manuela Aristizábal Zapata · 1094892680

Sebastián José Lesmes Muñoz · 100677504

Docente: Carolina Londoño Idárraga

Armenia, octubre de 2026

1. Resumen

Todas las pruebas se ejecutaron con el script `08_pruebas_entrega2.sql` en el esquema **TURISMOUQ** de Oracle Database 26ai Free (base conectable FREEPDB1), desde SQL Developer, el 29 de octubre de 2026. Cada caso compara el resultado obtenido con el esperado e imprime PASA o FALLA. Resultado: **39 casos PASA y 0 FALLA**. La salida completa está en `evidencia_entrega2.txt`; las tablas de resultados de las consultas se muestran como capturas.

Componente	Peso	Objeto	Casos	Resultado
fn_valor_estadia	20 %	`pkg_reservas.fn_valor_estadia`	8	PASA
sp_crear_reserva	15 %	`pkg_reservas.sp_crear_reserva`	11	PASA
Cursor liquidación mensual	15 %	`pkg_reservas.sp_liquidacion_mensual`	5	PASA
Paquete	10 %	`PKG_RESERVAS`	VALID	PASA
Disparador de sentencia	10 %	`trg_auditoria_tarifa`	4	PASA
Disparador de fila	10 %	`trg_rh_sin_solape`	3	PASA
Tablespace	5 %	`TS_TURISMOUQ_HIST`	4	PASA
Vista materializada	15 %	`MV_OCUPACION_MENSUAL`	4	PASA

Orden de ejecución: `04_paquete_reservas.sql`, `05_disparadores.sql`, `06_tablespace_historico_SYSTEM.sql` (como SYSTEM), `07_historico_y_vista_materializada.sql` y `08_pruebas_entrega2.sql`, todos en el esquema TURISMOUQ salvo el 06. Las pruebas usan una habitación libre del 10 al 20 de diciembre de 2026 que el script elige solo (habitación 154 del alojamiento 14) y deshacen con ROLLBACK todo lo que crean, excepto la auditoría, la liquidación y el histórico, que son la evidencia.

```
Datos de prueba: habitación 154 (capacidad 3), segunda habitación 135, alojamiento 14, cliente 1
```

2. fn_valor_estadia (20 %)

La función suma, noche por noche, la tarifa de la temporada vigente cada día: CALENDARIO asigna cada fecha a una temporada y TARIFA da el valor de esa habitación en esa temporada. La noche de salida no se cobra. En el calendario cargado la temporada alta empieza el 15 de diciembre, así que una estadía del 12 al 17 de diciembre de 2026 cobra 3 noches en temporada MEDIA y 2 en ALTA. La segunda estadía cruza además el cambio de año, que son dos temporadas distintas.

```
Habitación 154 · 2026-12-12 -> 2026-12-17
Noche       Cat.   Temporada                                      Tarifa     Acumulado
2026-12-12  MEDIA  Media de comienzos de diciembre 2026          248,000       248,000
2026-12-13  MEDIA  Media de comienzos de diciembre 2026          248,000       496,000
2026-12-14  MEDIA  Media de comienzos de diciembre 2026          248,000       744,000
2026-12-15  ALTA   Vacaciones de fin de año 2026                309,000     1,053,000
2026-12-16  ALTA   Vacaciones de fin de año 2026                309,000     1,362,000
fn_valor_estadia = 1,362,000
```

```
Habitación 154 · 2025-12-30 -> 2026-01-02
Noche       Cat.   Temporada                                      Tarifa     Acumulado
2025-12-30  ALTA   Vacaciones de fin de año 2025                289,000       289,000
2025-12-31  ALTA   Vacaciones de fin de año 2025                289,000       578,000
2026-01-01  ALTA   Vacaciones de inicio de año 2026             309,000       887,000
fn_valor_estadia = 887,000
```

Casos verificados: los valores se comparan con las tarifas leídas directamente de la tabla, y las fechas inválidas deben lanzar el error propio correspondiente.

```
PASA   1.1 Cruce MEDIA->ALTA: 3 x 248000 + 2 x 309000 = 1362000
PASA   1.2 Una sola temporada: 3 x 248000 = 744000
PASA   1.3 Cruce de año: 2 x 289000 (2025) + 1 x 309000 (2026) = 887000
PASA   1.4 Una noche = tarifa de esa noche (309000)
PASA   1.5 Check-out = check-in  -> ORA-20102: El check-out (2026-12-12) debe ser posterior al
check-in (2026-12-12).
PASA   1.6 Check-out < check-in  -> ORA-20102: El check-out (2026-12-12) debe ser posterior al
check-in (2026-12-17).
PASA   1.7 Fuera del calendario   -> ORA-20106: Solo 2 de 4 noches tienen tarifa: el rango sale del
calendario de temporadas cargado.
PASA   1.8 Habitación inexistente -> ORA-20104: La habitación -1 no existe.
```

3. sp_crear_reserva (15 %)

El procedimiento valida y lanza excepciones propias con RAISE_APPLICATION_ERROR. Las tres que pide el enunciado son: **-20101** habitación no disponible, **-20102** check-out anterior o igual al check-in y **-20103** capacidad excedida. También valida el cliente (-20105), el check-in en el pasado (-20107) y la habitación (-20104). Bloquea la fila de la habitación con `SELECT ... FOR UPDATE` y, ante cualquier error, deshace solo su propio trabajo con `ROLLBACK TO SAVEPOINT`. El caso 2.9 demuestra que los intentos fallidos no dejan filas a medias. El paquete también declara las excepciones con nombre (caso 2.3).

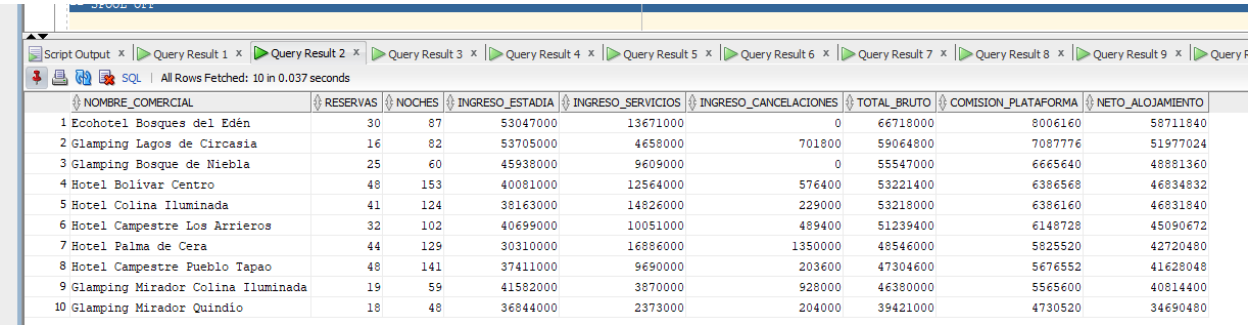
```
PASA   2.1 Reserva creada: id 30020, estado PENDIENTE, valor 1362000 (= fn_valor_estadia)
PASA   2.2 No disponible -> ORA-20101: Habitación 154 no disponible del 2026-12-14 al 2026-12-16: se
cruza con la reserva 30020 (2026-12-12 a 2026-12-17).
PASA   2.3 Capturada como pkg_reservas.e_no_disponible
PASA   2.4 Check-out < check-in -> ORA-20102: El check-out (2026-12-18) debe ser posterior al check-
in (2026-12-20).
PASA   2.5 Check-out = check-in -> ORA-20102: El check-out (2026-12-18) debe ser posterior al check-
in (2026-12-18).
PASA   2.6 Capacidad excedida -> ORA-20103: Capacidad excedida: 4 huéspedes y la habitación 135
admite máximo 3.
PASA   2.7 Cliente inexistente -> ORA-20105: El cliente -1 no existe.
PASA   2.8 Check-in en el pasado -> ORA-20107: No se puede reservar con check-in en el pasado (2025-
06-01).
PASA   2.9 Los 7 intentos fallidos no dejaron filas: reservas antes 30001 = después 30001 (incluye
la de 2.1)
PASA   2.10 Reserva 30020 con 2 habitaciones, rango hasta 2026-12-18
PASA   2.11 Otro alojamiento -> ORA-20110: La habitación 7 no pertenece al alojamiento de la reserva
30020.
```

Todo lo creado en esta prueba se deshizo con ROLLBACK al final del bloque.

4. Cursor explícito con parámetros: liquidación mensual (15 %)

`sp_liquidacion_mensual(p_anio, p_mes, p_id_alojamiento)` recorre el cursor `c_liquidacion(p_ini, p_fin, p_id_aloj)` con OPEN, FETCH, EXIT WHEN %NOTFOUND y CLOSE. Por cada alojamiento calcula el ingreso de las estadías COMPLETADAS cuyo check-out cae en el mes, valoradas con los importes congelados al reservar; le suma los servicios y lo retenido por cancelaciones; descuenta la comisión de la plataforma (12 %) y guarda el resultado en LIQUIDACION_MENSUAL. Liquidación de diciembre de 2025 (60 alojamientos; aquí se muestra el total):

```
LIQUIDACIÓN 2025-12 · comisión plataforma 12 %
Municipio    Alojamiento                          Res.  Noches          Bruto      Comisión          Neto
   ... 60 filas, una por alojamiento ...
TOTAL (60 alojamientos)                                          1,302,206,000   156,264,720  1,145,941,280
```



Script Output | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7 | Query Result 8 | Query Result 9

SQL | All Rows Fetched: 10 in 0.037 seconds

	NOMBRE_COMERCIAL	RESERVAS	NOCHES	INGRESO_ESTADIA	INGRESO_SERVICIOS	INGRESO_CANCELACIONES	TOTAL_BRUTO	COMISION_PLATAFORMA	NETO_ALOJAMIENTO
1	Ecohotel Bosques del Edén	30	87	53047000	13671000	0	66718000	8006160	58711840
2	Glamping Lagos de Circasia	16	82	53705000	4658000	701800	59064800	7087776	51977024
3	Glamping Bosque de Niebla	25	60	45938000	9609000	0	55547000	6665640	48881360
4	Hotel Bolívar Centro	48	153	40081000	12564000	576400	53221400	6386568	46834832
5	Hotel Colina Iluminada	41	124	38163000	14826000	229000	53218000	6386160	46831840
6	Hotel Campestre Los Arrieros	32	102	40699000	10051000	489400	51239400	6148728	45090672
7	Hotel Palma de Cera	44	129	30310000	16886000	1350000	48546000	5825520	42720480
8	Hotel Campestre Pueblo Tapao	48	141	37411000	9690000	203600	47304600	5676552	41628048
9	Glamping Mirador Colina Iluminada	19	59	41582000	3870000	928000	46380000	5565600	40814400
10	Glamping Mirador Quindío	18	48	36844000	2373000	204000	39421000	4730520	34690480

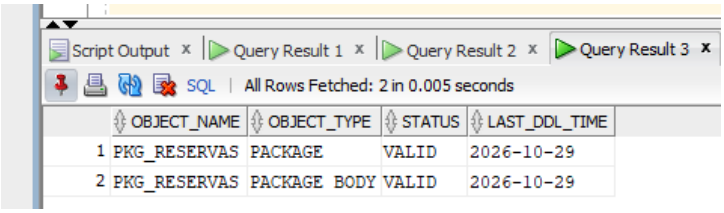
Figura 1. Los 10 alojamientos con mayor neto a liquidar en diciembre de 2025 (LIQUIDACION_MENSUAL).

Verificaciones: la suma por estadía cuadra con una consulta independiente, las cuentas de cada fila son consistentes, la re-ejecución para un solo alojamiento reemplaza solo esa fila y un mes inválido lanza el error propio.

```
PASA   5.1 Una fila por alojamiento: 60
PASA   5.2 Ingreso por estadía cuadra con la consulta directa: 1034590000 = 1034590000
PASA   5.3 bruto = estadía + servicios + cancelaciones y neto = bruto - comisión en todas las filas
PASA   5.4 Re-ejecución para el alojamiento 14: siguen 60 filas, sin duplicados
PASA   5.5 Periodo inválido -> ORA-20113: Periodo inválido: año 2025, mes 13.
```

5. Paquete PKG_RESERVAS (10 %)

La especificación expone la función, los procedimientos, la constante de comisión y las excepciones con nombre. El cuerpo contiene además el cursor parametrizado y el procedimiento privado `validar_linea`, que comparten la creación de reservas y la adición de habitaciones.

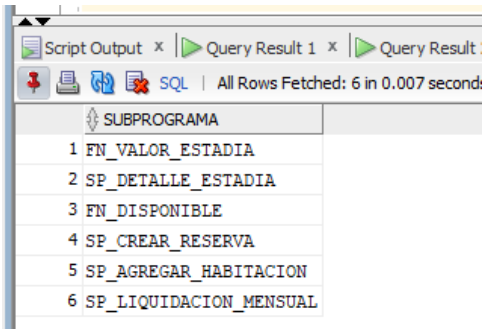


Script Output | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 2 in 0.005 seconds

	OBJECT_NAME	OBJECT_TYPE	STATUS	LAST_DDL_TIME
1	PKG_RESERVAS	PACKAGE	VALID	2026-10-29
2	PKG_RESERVAS	PACKAGE BODY	VALID	2026-10-29

Figura 2. Especificación y cuerpo de PKG_RESERVAS compilados (VALID).



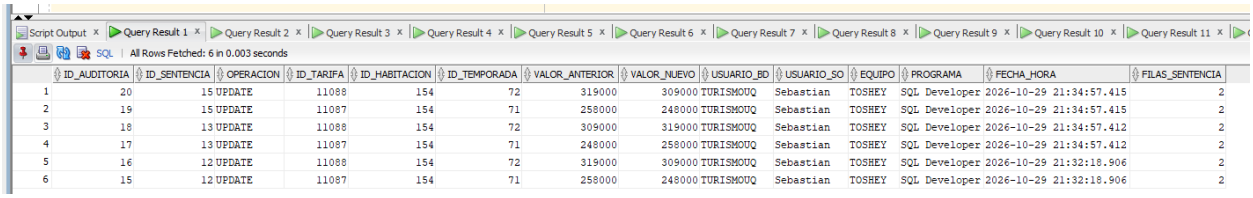
	SUBPROGRAMA
1	FN_VALOR_ESTADIA
2	SP_DETALLE_ESTADIA
3	FN_DISPONIBLE
4	SP_CREAR_RESERVA
5	SP_AGREGAR_HABITACION
6	SP_LIQUIDACION_MENSUAL

Figura 3. Subprogramas públicos del paquete (USER_PROCEDURES).

6. Disparador de sentencia: auditoría de TARIFA (10 %)

`trg_auditoria_tarifa` registra quién (usuario Oracle, usuario del sistema operativo, equipo y programa), cuándo (fecha y hora), y el valor anterior y el nuevo de cada tarifa modificada. Un disparador de sentencia no puede leer :OLD ni :NEW, por eso es un disparador **compuesto**: la sección AFTER EACH ROW solo guarda los valores en memoria, y la sección **AFTER STATEMENT** escribe la auditoría una sola vez al terminar la sentencia, con el número de filas que cambió. Las filas de una misma sentencia comparten `id_sentencia`.

```
PASA   4.1 UPDATE de 2 tarifas -> 2 filas de auditoría de 1 sentencia, con usuario, hora, valor
anterior y nuevo
PASA   4.2 fn_valor_estadia pasó de 1362000 a 1412000 (+5 noches x 10.000)
PASA   4.3 UPDATE que no cambia el valor: 0 filas de auditoría
PASA   4.4 Tarifas restauradas: valor vuelve a 1362000
```



	ID_AUDITORIA	ID_SENTENCIA	OPERACION	ID_TARIFA	ID_HABITACION	ID_TEMPORADA	VALOR_ANTERIOR	VALOR_NUEVO	USUARIO_BD	USUARIO_SO	EQUIPO	PROGRAMA	FECHA_HORA	FILAS_SENTENCIA
1	20	15	UPDATE	11088	154	72	319000	309000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:34:57.415	2
2	19	15	UPDATE	11087	154	71	258000	248000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:34:57.415	2
3	18	13	UPDATE	11088	154	72	309000	319000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:34:57.412	2
4	17	13	UPDATE	11087	154	71	248000	258000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:34:57.412	2
5	16	12	UPDATE	11088	154	72	319000	309000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:32:18.906	2
6	15	12	UPDATE	11087	154	71	258000	248000	TURISMOUQ	Sebastian	TOSHEY	SQL Developer	2026-10-29 21:32:18.906	2

Figura 4. AUDITORIA_TARIFA: cada UPDATE de 2 tarifas queda como 2 filas con el mismo id_sentencia.

En la captura, la sentencia 13 sube las dos tarifas 10.000 pesos (caso 4.1) y la 15 las restaura (caso 4.4). La sentencia 14, el UPDATE que no cambia el valor (caso 4.3), consumió un número pero no dejó filas, porque un cambio sin diferencia no se audita. La sentencia 12 es la restauración de una ejecución anterior de las pruebas.

7. Disparador de fila: solape de reservas (10 %)

`trg_rh_sin_solape` (BEFORE INSERT … FOR EACH ROW sobre RESERVA_HABITACION) rechaza con **-20120** una línea que comparta alguna noche con otra reserva no cancelada de la misma habitación. Para probarlo con un caso real, la línea se inserta **directamente en la tabla**, sin pasar por el paquete: el disparador la bloquea por su cuenta (3.1). Llegar el mismo día en que sale el huésped anterior no es un solape (3.2), y una reserva cancelada no ocupa la habitación (3.3).

```
Reserva A = 30021 (habitación 154, 2026-12-12 a 2026-12-17)
PASA   3.1 Solape rechazado -> ORA-20120: trg_rh_sin_solape: la habitación 154 ya está reservada
entre el 2026-12-15 y el 2026-12-19 por la reserva 30021.
ORA-06512: at "TURISMOUQ.TRG_RH_SIN_SOLAPE", line 26
ORA-04088: error during execution of trigger 'TURISMOUQ.TRG_RH_SIN_SOLAPE'
PASA   3.2 Llegada el día de salida de A: aceptada
PASA   3.3 Línea de reserva cancelada que se cruza: aceptada
```

La pila de errores muestra que el rechazo viene del disparador TURISMOUQ.TRG_RH_SIN_SOLAPE. Al final se hizo ROLLBACK.

8. Tablespace del histórico de reservas (5 %)

8.1 Justificación

Las reservas cerradas (completadas o canceladas de periodos anteriores) se usan de forma muy distinta a las reservas vivas, y por eso van en un tablespace propio:

- **Patrón de uso:** el histórico solo crece y casi no se lee; las reservas vivas se insertan y actualizan todo el día. Separarlos evita que compitan y permite PCTFREE `0` en el histórico, que nunca se actualiza.
- **Crecimiento controlado:** el histórico tiene su propio límite (AUTOEXTEND hasta 1 GB), así que no puede llenar USERS, donde viven las tablas operativas, y detener las ventas.
- **Respaldo:** un periodo cerrado no cambia, así que su tablespace se respalda con menos frecuencia o se pasa a READ ONLY al cerrar un año.
- **Ubicación física:** su datafile puede ir en un disco más barato sin tocar el de la operación.

8.2 Creación (como SYSTEM)

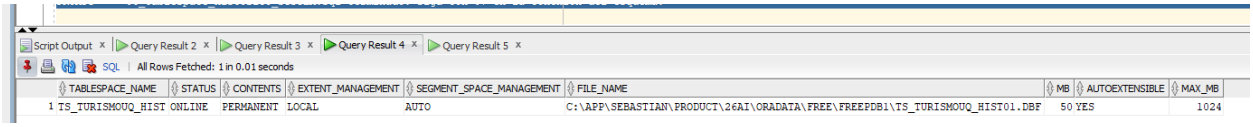


Figura 5. TS_TURISMOUQ_HIST: tablespace local, 50 MB iniciales, autoextensible hasta 1024 MB.

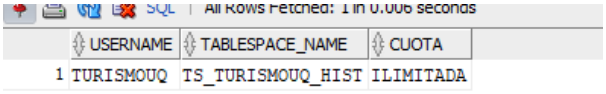
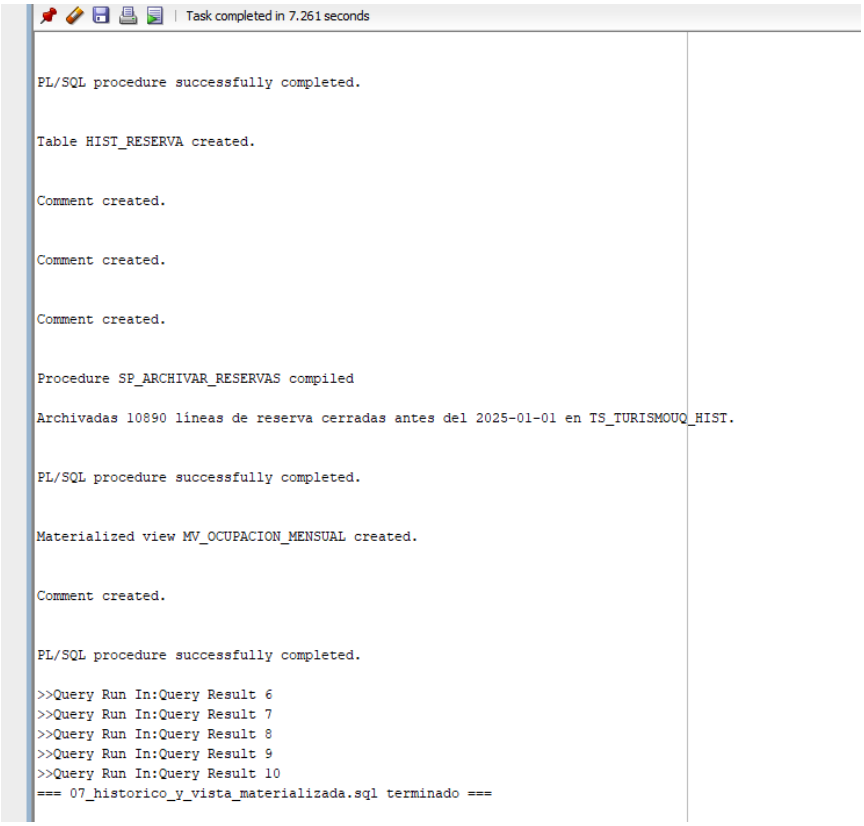


Figura 6. Cuota ilimitada del esquema TURISMOUQ en el nuevo tablespace.

8.3 Histórico en el tablespace



```
Task completed in 7.261 seconds

PL/SQL procedure successfully completed.

Table HIST_RESERVA created.

Comment created.

Comment created.

Comment created.

Procedure SP_ARCHIVAR_RESERVAS compiled

Archivadas 10890 líneas de reserva cerradas antes del 2025-01-01 en TS_TURISMOUQ_HIST.

PL/SQL procedure successfully completed.

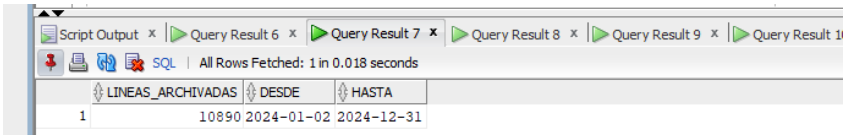
Materialized view MV_OCUPACION_MENSUAL created.

Comment created.

PL/SQL procedure successfully completed.

>>Query Run In:Query Result 6
>>Query Run In:Query Result 7
>>Query Run In:Query Result 8
>>Query Run In:Query Result 9
>>Query Run In:Query Result 10
=== 07_historico_y_vista_materializada.sql terminado ===
```

Figura 7. Ejecución de 07: HIST_RESERVA, el procedimiento de archivo y la vista materializada creados; 10.890 líneas de 2024 archivadas.



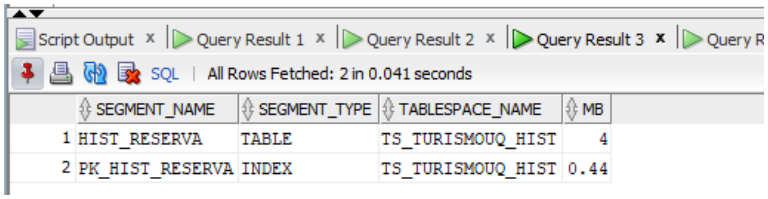
Script Output | Query Result 6 | Query Result 7 | Query Result 8 | Query Result 9 | Query Result 10

SQL | All Rows Fetched: 1 in 0.018 seconds

	LINEAS_ARCHIVADAS	DESDE	HASTA
1	10890	2024-01-02	2024-12-31

Figura 8. Carga inicial del histórico: año 2024 completo.

```
PASA   7.1 HIST_RESERVA está en TS_TURISMOUQ_HIST
PASA   7.2 Re-archivar 2024 no duplica: 16988 = 16988
PASA   7.3 Archivo hasta 2025-07-01: faltaban 0, histórico 16988 -> 16988
PASA   7.4 No queda ninguna línea cerrada antes del 2025-07-01 sin archivar
```



Script Output | Query Result 1 | Query Result 2 | Query Result 3 | Query R

SQL | All Rows Fetched: 2 in 0.041 seconds

	SEGMENT_NAME	SEGMENT_TYPE	TABLESPACE_NAME	MB
1	HIST_RESERVA	TABLE	TS_TURISMOUQ_HIST	4
2	PK_HIST_RESERVA	INDEX	TS_TURISMOUQ_HIST	0.44

Figura 9. La tabla HIST_RESERVA y su índice ocupan espacio en TS_TURISMOUQ_HIST.

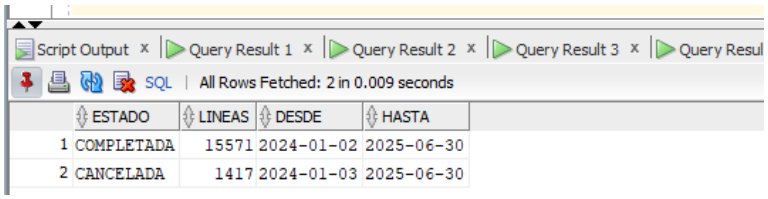


Figura 10. Contenido del histórico tras archivar hasta el 30 de junio de 2025: 16.988 líneas.

9. Vista materializada MV_OCUPACION_MENSUAL (15 %)

Precalcula, para cada alojamiento y mes de 2024 a 2026 (60 × 36 = 2.160 filas), las habitaciones, las noches disponibles, las noches ocupadas y el porcentaje de ocupación.

9.1 Política de refresco: COMPLETE ON DEMAND, programada cada noche a las 2 a. m.

- **No FAST (incremental):** la consulta cruza las reservas con el calendario por rango de fechas y agrega sobre ese cruce, algo que el refresco rápido no admite.
- **No ON COMMIT:** recalcularía la ocupación de todo el departamento en cada transacción de reserva y volvería lenta la venta.
- **Diario es suficiente:** la ocupación mensual es un indicador de gestión que tolera un día de retraso. A las 2 a. m. casi no hay reservas en curso, y cuando hace falta el dato al instante se refresca a mano con `DBMS_MVIEW.REFRESH`.
- **atomic_refresh => TRUE:** quien consulte durante el refresco ve los datos anteriores, nunca una vista vacía.

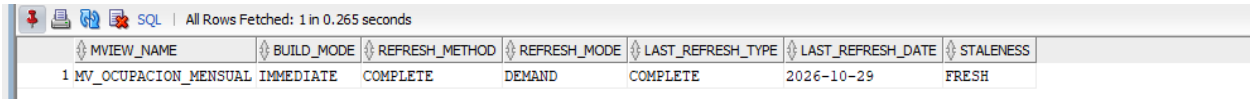


Figura 11. Definición registrada: BUILD IMMEDIATE, REFRESH COMPLETE ON DEMAND, estado FRESH.

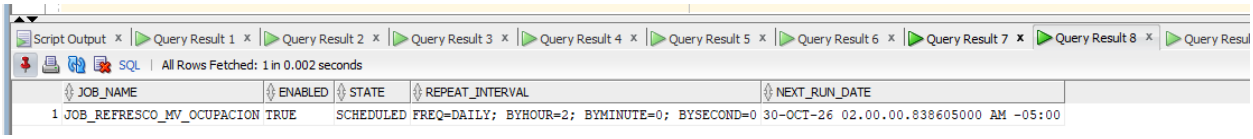


Figura 12. Job de DBMS_SCHEDULER que la refresca cada día a las 2:00 a. m.

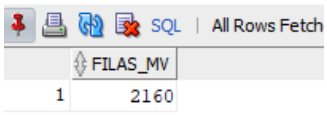
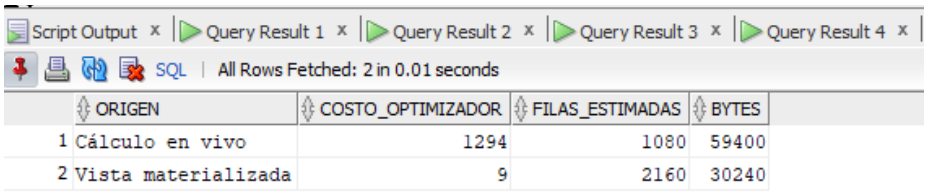


Figura 13. Filas de la vista: 60 alojamientos × 36 meses.

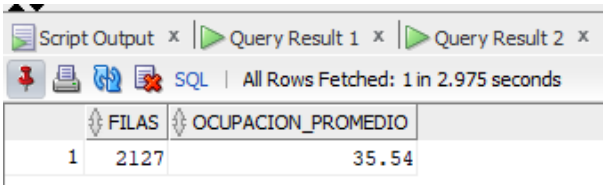
9.2 Beneficio de precalcular

El optimizador estima un costo de **1.294** para calcular la ocupación en vivo y de **9** para leerla de la vista, unas 140 veces menos trabajo. Los tiempos de la misma consulta van en la misma dirección: 2,975 s en vivo contra 0,003 s desde la vista.



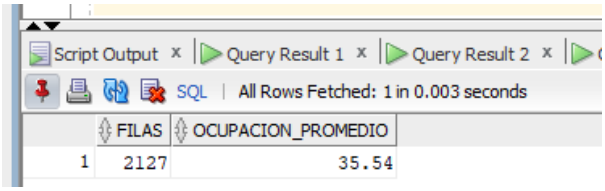
	ORIGEN	COSTO_OPTIMIZADOR	FILAS_ESTIMADAS	BYTES
1	Cálculo en vivo	1294	1080	59400
2	Vista materializada	9	2160	30240

Figura 14. Costo del plan de ejecución (EXPLAIN PLAN): cálculo en vivo vs. vista materializada.



	FILAS	OCUPACION_PROMEDIO
1	2127	35.54

Figura 15. Ocupación calculada en vivo sobre las reservas: 2,975 s.



	FILAS	OCUPACION_PROMEDIO
1	2127	35.54

Figura 16. El mismo resultado leído de la vista materializada: 0,003 s.

9.3 Comportamiento ON DEMAND

Recién refrescada, la vista coincide con el cálculo en vivo (8.1). Al confirmar una nueva reserva de 5 noches, la vista sigue mostrando el valor anterior, que es lo esperado en ON DEMAND (8.2). Después de `DBMS_MVIEW.REFRESH` muestra el nuevo valor (8.3). La reserva de prueba se eliminó y la vista se volvió a refrescar (8.4).

```
PASA   8.1 Recién refrescada coincide con el cálculo en vivo: 75
PASA   8.2 Tras confirmar la reserva 30024: vista 75 (desactualizada), en vivo 80
PASA   8.3 Después de DBMS_MVIEW.REFRESH la vista muestra 80
PASA   8.4 Reserva de prueba eliminada y vista refrescada: 75
```